Thamill Indian B U
Cohort-15 (AI Engineer)

Day 05 Engineering Reflection

AI ENGINEERING PROGRAM | DOCUMENT PARSING & CHUNKING ENGINE

Part 1 : Build Evidence

Parsed & Cleaned Source Output: Ingested `sample_doc.md` (2,375 characters), normalizing whitespaces while preserving structural headings, markdown tables, and bulleted item hierarchies.

Dual-Approach Chunking Output:

- **Fixed-Size Strategy:** Emitted 7 uniform chunks (400-char window with 50-char overlap).
- **Structural Strategy:** Emitted 7 semantic chunks bounded by markdown headers (range: 214–550 chars).

Metadata Integration & Quality Checks: Tagged each chunk with `source_document`, sequential `chunk_index`, hierarchical `section_heading` paths, and strategy tags. Fixed-size chunking triggered boundary warnings on multi-sentence policies, whereas structural chunking maintained complete semantic integrity.

Part 2: Experimentation & Comparative Analysis

Fixed-Size Approach	Structural Approach
• Size: 400 chars | Overlap: 50 chars	• Boundary: Logical markdown headers
• 11.2% character overlap fraction	• Zero arbitrary mid-sentence cuts
• Fragmented multi-sentence rules across windows	• Retained complete rule and exception blocks

- **Which two chunking approaches did you compare?** We compared Fixed-Size Window Chunking (with character overlap) against Markdown Header-Aware Structural (Semantic) Chunking.
- **What size/boundary/overlap choices did you make?**
  The fixed-size approach used a target window of 400 characters with a 50-character overlap (yielding an 11.2% character overlap fraction), while the structural approach used natural markdown section headers as semantic boundaries with zero artificial overlap.
- **Which example most clearly showed a difference?**
  The 550-character temporary access policy rule clearly showed the difference; fixed-size chunking fragmented it across three separate windows, whereas structural chunking kept the complete rule and its expiry exception unified within a single chunk.
- **Where did context get lost?** Context got lost in fixed-size chunking when arbitrary character boundaries split multi-sentence rules mid-sentence or separated technical details from their parent section headers.
- **Where did overlap create unnecessary duplication?** Fixed-size overlap created unnecessary character duplication across neighboring pairs by repeating 50-character substrings, adding redundant character overhead without guaranteeing rule completeness.

Part 3: Engineering Reasoning

> **Recommended Approach for Day 06:** Structural / heading-aware chunking combined with rich metadata enrichment. This preserves semantic boundaries and prevents policy fragmentation during retrieval testing.

- **Trade-off Accepted:** Accepting variable chunk payloads (21 4–550 characters) in exchange for complete semantic units and zero fragmented policy clauses.
- **Why Fixed Size is Suboptimal:** Documents combine prose, lists, and tables. Rigid window sizes ignore semantic structure, fracturing multi-sentence rules and separating critical operational details from governing headings.
- **Essential Metadata Fields:** `source_document`, hierarchical `section_heading` paths, and `chunk_index` are critical for downstream filtering, citations, access control, and debugging.

Part 4: FDE Reflection

- **Sources & Ownership:** Ask customers who owns repository sources and how deprecated or conflicting document versions are filtered prior to pipeline ingestion.
- **Document Freshness:** Outdated operational policies can cause retrieval engines to surface obsolete instructions. Automated timestamp tracking is vital.
- **Access Security:** Role-Based Access Control (RBAC) metadata must be attached to chunks to guarantee users only retrieve authorized information.
- **Definition of Success:** High semantic coherence (no split rules or orphaned headers), complete metadata tagging, and zero loss of critical operational details.

Part 5: One Improvement

Next 30-Minute Enhancement: Implement intelligent table and list serialization (converting markdown tables into structured text key-value representations within chunk payloads) to improve structured data readability during structural chunking.